Интегрирование дифференциальных уравнений

Основы MATLAB

Юдинцев В. В.

Кафедра теоретической механики

2 сентября 2022 г.

САМАРСКИЙ УНИВЕРСИТЕТ
SAMARA UNIVERSITY

Содержание

1. Интегрирование ОДУ
2. Задача баллистики
3. Движение точки в центральном поле
4. Пересчёт координат из геоцентрической СК в орбитальную подвижную

Интегрирование ОДУ

Дифференциальные уравнения

$$\begin{cases} \dfrac{dq_1}{dt} = f_1(t, q_1, q_2, \ldots, q_n), \\ \dfrac{dq_2}{dt} = f_2(t, q_1, q_2, \ldots, q_n), \\ \ldots \\ \dfrac{dq_n}{dt} = f_n(t, q_1, q_2, \ldots, q_n). \end{cases} \quad \Rightarrow \quad \frac{d\boldsymbol{q}}{dt} = \boldsymbol{f}(t, \boldsymbol{q})$$

где

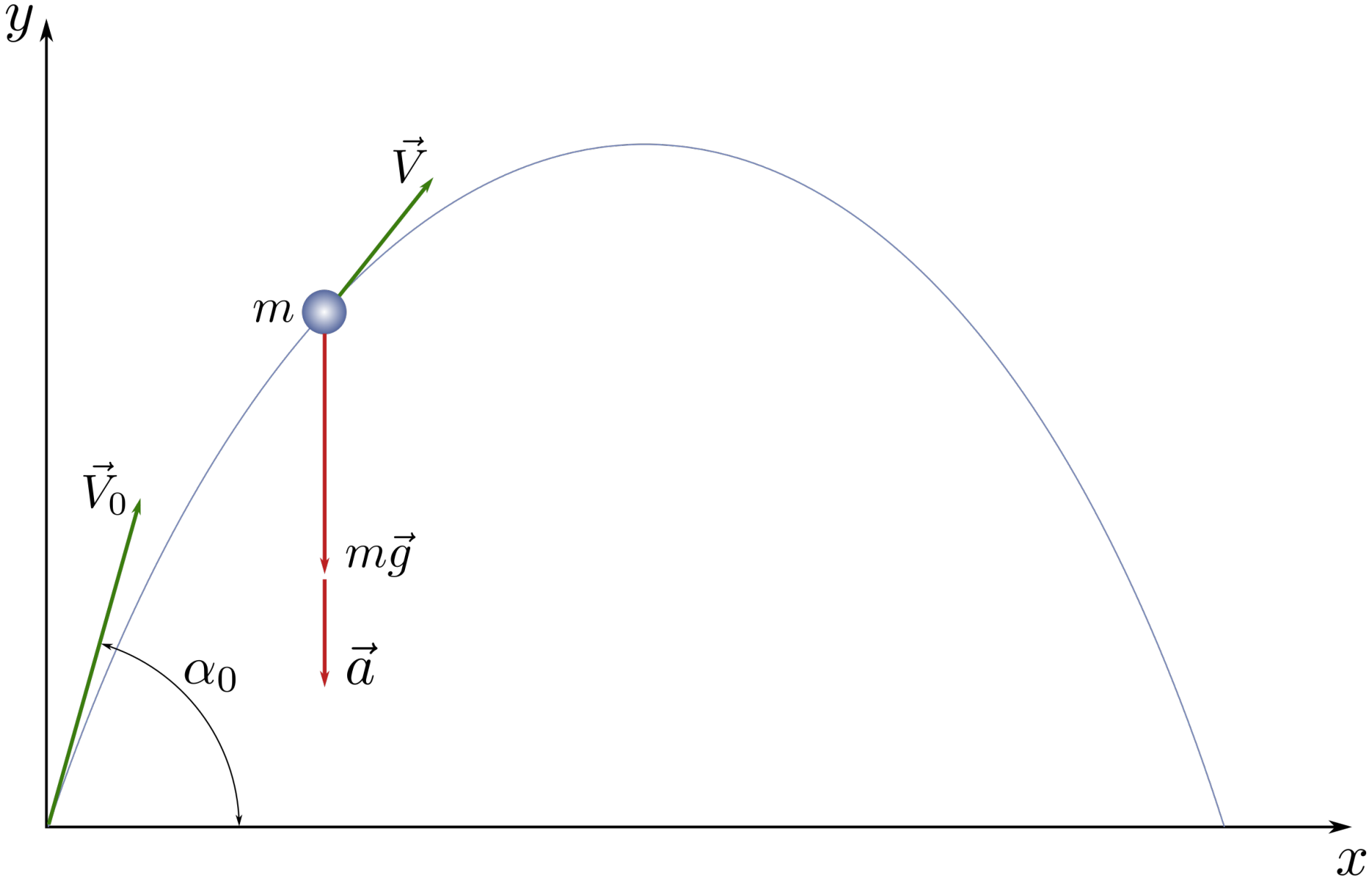
- $\boldsymbol{q} = (q_1, q_2, \ldots, q_n)^T$ – вектор состояния;
- $\boldsymbol{f}(t, \boldsymbol{q}) = (f_1, f_2, \ldots, f_n)^T$ – вектор-функция правых частей.

Алгоритм

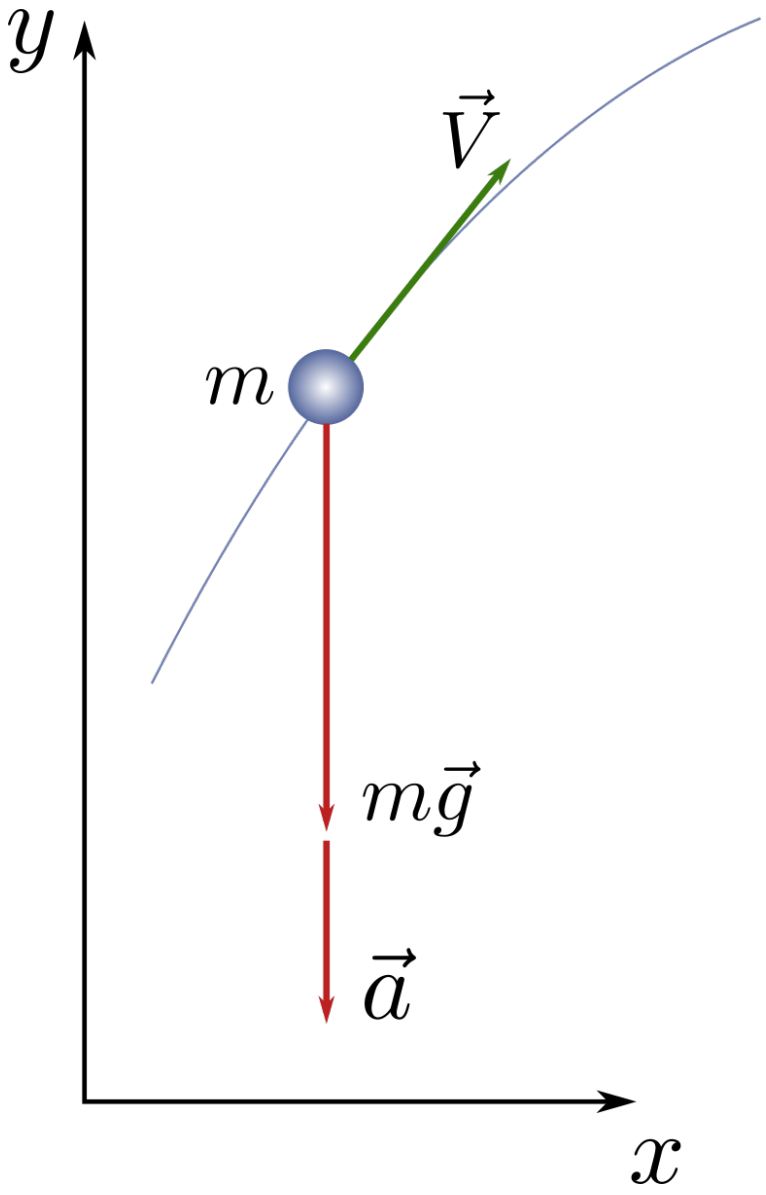
- Приведение уравнений к системе дифференциальных уравнений первого порядка.
- Разработка файл-функции правой части $\boldsymbol{f}(t, \boldsymbol{q})$.
- Разработка файл-скрипта запуска процесса интегрирвования ДУ.

Задача баллистики

Пример: задача баллистики



Уравнения движения



Векторное уравнение

$$m\vec{a} = \vec{F}_g = m\vec{g}.$$

Проекции векторного уравнения на оси координат:

$$ma_x = m\frac{d^2x}{dt^2} = 0,$$
$$ma_y = m\frac{d^2y}{dt^2} = -mg.$$

Дифференциальные уравнения движения

Приведение к системе дифференциальных уравнений первого порядка.

$$\begin{cases} m\dfrac{d^2x}{dt^2} = 0, \\ m\dfrac{d^2y}{dt^2} = -mg. \end{cases} \quad \Rightarrow \quad \begin{cases} \dfrac{dx}{dt} = V_x, \\ \dfrac{dV_x}{dt} = 0, \\ \dfrac{dy}{dt} = V_y, \\ \dfrac{dV_y}{dt} = -g. \end{cases} \quad \Rightarrow \quad \frac{d\boldsymbol{q}}{dt} = \boldsymbol{f}(\boldsymbol{q}, t)$$

где

$$\boldsymbol{q} = (x, V_x, y, V_y)^T, \quad \boldsymbol{f}(\boldsymbol{q}, t) = (V_x, 0, V_y, -g)^T$$

ODE файл – файл-функция правых частей

Файл в котором вычисляется правая часть дифференциального уравнения или системы уравнений:

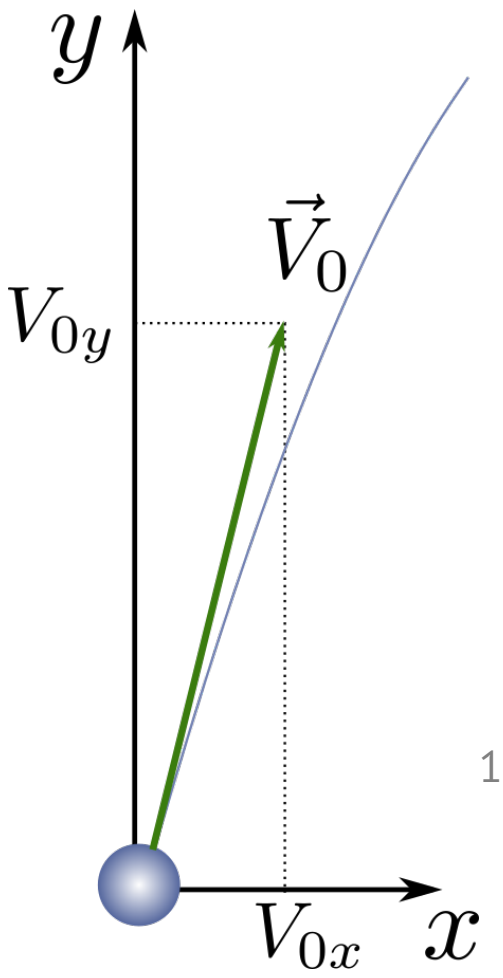
$$\mathbf{q} \Rightarrow \boxed{\dot{\mathbf{q}} = \mathbf{f}(\mathbf{q}, t)} \Rightarrow \dot{\mathbf{q}}$$

где $\mathbf{q}$ – вектор зависимых переменных, t – время.
Файл-функция `dqdt.m`

```
function dq = dqdt(t,q)
  g = 9.81;
  m = 1.00;
  x=q(1); Vx=q(2); y=q(3); Vy=q(4);
  dq = [Vx, 0, Vy, -g];
```

Интегрирование

Файл-скрипт основной программы (main.m)




```
% Начальные условия
V0 = 10;
a  = pi/4;
q0 = [0, V0*cos(a), 0, V0*sin(a)];

% Интервал интегрирования
tspan = [0 5];

% Запуск процесса интегрирования
[t, q] = ode45(@dqdt, tspan, q0);
```

Методы интегрирования

Интегратор	Описание
`ode45`	Метод Рунге-Кутты 4 и 5 порядков с автоматическим выбором шага
`ode23`	Метод Рунге-Кутты 2 и 3 порядков с автоматическим выбором шага
`ode113`	Многошаговый метод Адамса: метод с предсказанием и коррекцией
`ode15s`	Многошаговый метод для интегрирования жестких систем
`ode23s`	Одношаговый метод Розенброка второго порядка для жестких систем

Результаты интегрирования

```
[t, q] = ode45(@dqdt, tspan, q0);
```

`t` – столбец независимой переменной (время)

$$t = \begin{bmatrix} t_0 \\ t_1 \\ \dots \\ t_k \end{bmatrix}$$

`q` – матрица зависимых переменных

$$q = \begin{bmatrix} x_0 & V_{x0} & y_0 & V_{y0} \\ x_1 & V_{x1} & y_1 & V_{y1} \\ \dots & & & \\ x_k & V_{xk} & y_k & V_{yk} \end{bmatrix}$$

Управление процессом интегрирования

Расширенная версия запуска интегрирования д.у.:

```
[t, q] = ode45( @dqdt, tspan, q0, options);
```

До запуска интегрирования заполняется специальная структура `options` при помощи функции `odeset`:

```
options = odeset('Парам. 1', Значение, 'Парам. 2', ...);
[t, q] = ode45( @dqdt, tspan, q0, options);
```

Точность решения

- `'RelTol'`: относительная погрешность. **Число** - значение погрешности для всех переменных, или **вектор**, задающий погрешность для каждой переменной (по умолчанию 10^{-3})
- `'AbsTol'`: абсолютная погрешность. Число или вектор (по умолчанию 10^{-6})

```
options = odeset('RelTol',1e-4,'AbsTol',1e-8);
[t, q] = ode45( @dqdt, tspan , q0, options );
```

Графики

```
[t, q] = ode45( @dqdt, tspan , q0);
```

График изменения высоты по времени

```
plot(t, q(3,:));
xlabel('t, c');
ylabel('h, m');
```

Траектория

```
plot(q(1,:), q(3,:));
xlabel('x, m');
ylabel('y, m');
```

События: параметр 'Events'

Для остановки процесса интегрирования при выполении некоторого условия используется функция событий, которая передаётся интегратору:

```
options = odeset('Events',@events);
```

```
V0 = 1;
a  = pi/4;
q01 = [0,V0*cos(a),0,V0*sin(a)];
[t1,q1] = ode45(@dqdt,[0 20],q01);
```

Пример файл-функции событий events.m

$$\boldsymbol{q} = (x,\, V_x,\, y,\, V_y)^T$$

```
function [value,isterminal,direction]=events(t, q)
 % Следим за изменением переменной y
 value = q(3);
 % Остановить процесс,
 isterminal= 1;
 % если q(3) станет равна нулю и будет убывать
 direction =-1;
```

Файл-функция для остановки процесса интегрирования (isterminal=1) при пересечении нуля функцией $y(t)$ (value = q(3)) при её убывании (direction = -1).

Падение шарика на упругую поверхность

```
options = odeset('Events',@events);
v0 = 1;
q01 = [0,v0*cos(pi/4),0,v0*sin(pi/4)];
[t1,q1] = ode45(@dqdt,[0 20],q01);
```

```
n1 = length(t1);
```

Перед интегрированием второго этапа меняем знак скорости V_y (шарик отскочил с в два раза меньшей скоростью):

```
q02    =  q1(n1,:);
q02(4)= -q02(4)*0.5;
```

Падение шарика на упругую поверхность

Интегрируем от t_{1k} до $t_{1k} + 20$

```
t0 = t1(n1); tk=t0+20;
[t2,q2] = ode45(@dqdt,[t0 tk],q02);
```

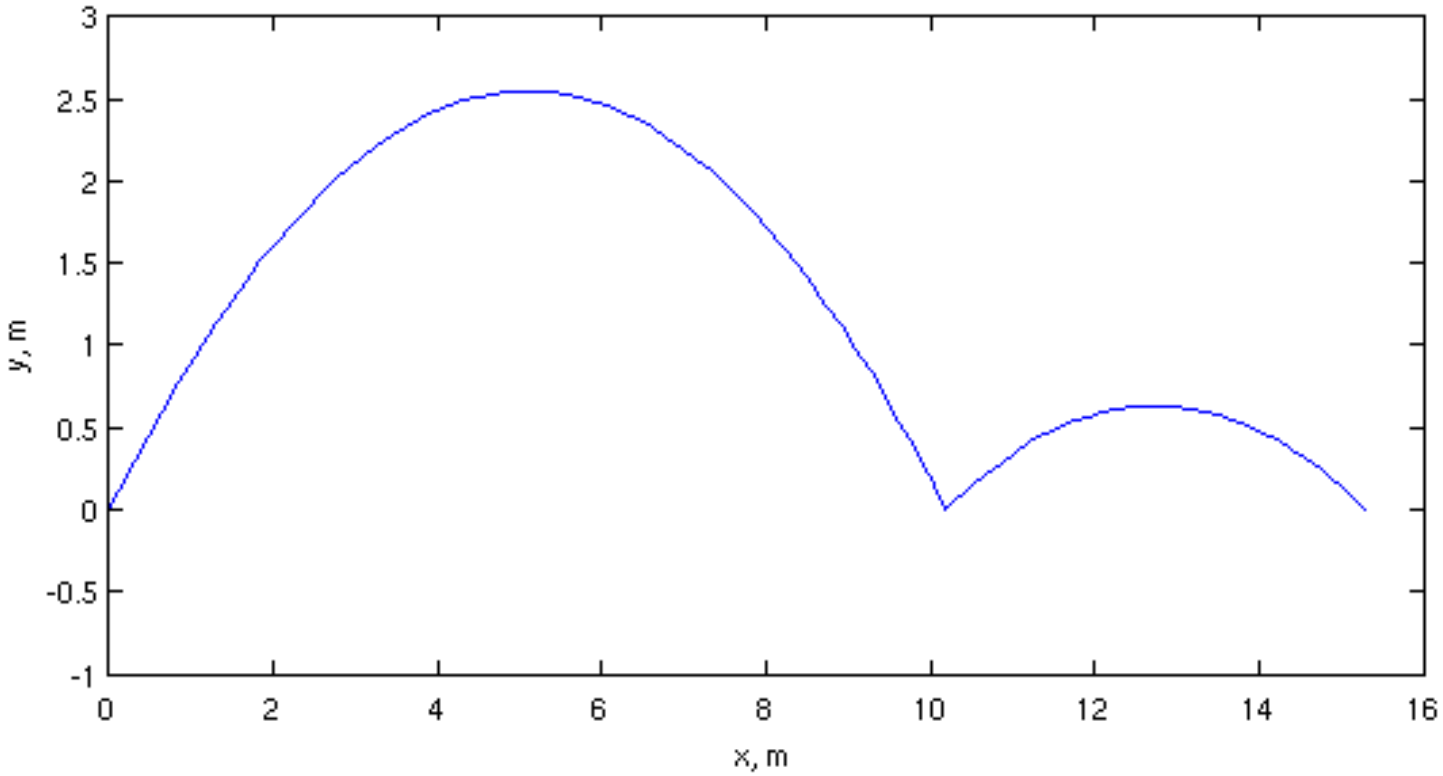
“Склеиваем” результаты двух этапов движения:

```
t=[t1;t2]; q=[q1;q2];
```

Падение шарика на упругую поверхность

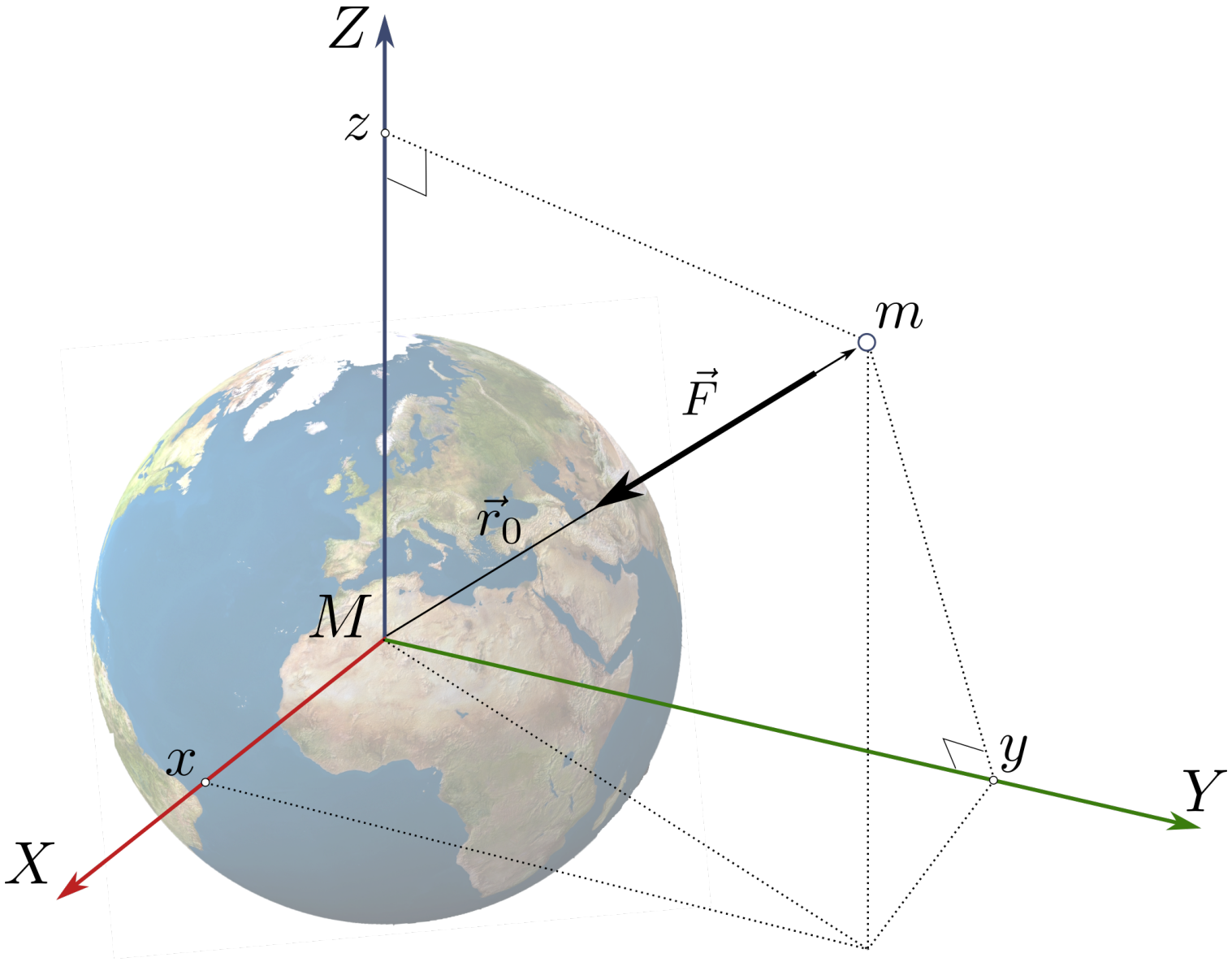
Построение траектории движения шарика:

```
plot(q(:,1),q(:,3));
```



Движение точки в центральном поле

Геоцентрическая инерциальная СК



Векторное уравнение движения

$$m\vec{a} = \vec{F}$$

Модуль силы притяжения Земли, действующей на спутник:

$$|\vec{F}| = F = G\frac{Mm}{r_0^2} = \mu\frac{m}{r_0^2},$$

где

$$\mu = GM \approx 3.986 \cdot 10^{14} \text{ м}^3/\text{с}^2$$

гравитационный параметр Земли

Координатная запись

$$ma_x = F_x = -F\cos(\widehat{X, -\vec{F}}) = -F\cos(\widehat{X, r_0}) = -F \cdot \frac{x}{r_0},$$

$$ma_y = F_y = -F\cos(\widehat{Y, -\vec{F}}) = -F\cos(\widehat{Y, r_0}) = -F \cdot \frac{y}{r_0},$$

$$ma_z = F_z = -F\cos(\widehat{Z, -\vec{F}}) = -F\cos(\widehat{Z, r_0}) = -F \cdot \frac{z}{r_0}.$$

Уравнения движения

$$\boxed{ma_x} = m\frac{d^2x}{dt^2} = F_x = -F\cdot\frac{x}{r_0} = -\mu\frac{m}{r_0^2}\,\frac{x}{r_0} = \boxed{-\mu\frac{m\cdot x}{r_0^3}},$$

$$\boxed{ma_y} = m\frac{d^2y}{dt^2} = F_y = -F\cdot\frac{y}{r_0} = -\mu\frac{m}{r_0^2}\,\frac{y}{r_0} = \boxed{-\mu\frac{m\cdot y}{r_0^3}},$$

$$\boxed{ma_z} = m\frac{d^2z}{dt^2} = F_z = -F\cdot\frac{z}{r_0} = -\mu\frac{m}{r_0^2}\,\frac{z}{r_0} = \boxed{-\mu\frac{m\cdot z}{r_0^3}}.$$

$$r_0 = \sqrt{x^2+y^2+z^2}$$

Приведение к ОДУ первого порядка

$$\begin{cases} ma_x = F_x, \\ ma_y = F_y, \\ ma_z = F_z. \end{cases} \quad \Rightarrow \quad \begin{cases} \dot{v}_x = F_x/m = -\mu\, x/r_0^3, \\ \dot{v}_y = F_y/m = -\mu\, y/r_0^3, \\ \dot{v}_z = F_z/m = -\mu\, z/r_0^3, \\ \dot{x} = v_x, \\ \dot{y} = v_y, \\ \dot{z} = v_z \end{cases}$$

Неизвестные функции:

$$v_x, v_y, v_z, x, y, z$$

Файл-функция правой части dqdt.m

Вектор состояния:

$$q = (v_x, v_y, v_z, x, y, z)$$

```
function dq = dqdt(t, q)
  global params;
  v = q(1:3);
  r = q(4:6);
  % Вектор ускорения
  a = - params.mu*r/norm(r,2)^3;
  % Возвращаем производную вектора q
  dq = [a,v];
```

Главный файл-скрипт orbitalMotion.m

```
% Гравитационный параметр Земли
params.mu=3.986e14;
% Радиус Земли
params.Rz = 6371000;
% Формируем вектор начальных условий
% для круговой орбиты высотой
h  = 300000;
% Орбитальная скорость
V  = sqrt(params.mu/(params.Rz+h));
q0 = [0;V;0;h+Rz;0;0];
% От 0 до одного орбитального периода
tk = 2*pi*(h+params.Rz)/V;
% Вызываем функцию интегрирования
[t,q] = ode45(@dqdt, q0, [0 tk]);
```

Файл-функция правой части dqdt.m

Вариант функции правых частей с передачей параметров

```
function dq = dqdt(t, q, params)
  v = q(1:3);
  r = q(4:6);
  % Вектор ускорения
  a = - params.mu*r/norm(r,2)^3;
  % Возвращаем производную вектора q
  dq = [a,v];
```

Главный файл-скрипт orbitalMotion.m

```
% Гравитационный параметр Земли
params.mu=3.986e14;
% Радиус Земли
params.Rz = 6371000;
% Формируем вектор начальных условий
% для круговой орбиты высотой
h  = 300000;
% Орбитальная скорость
V  = sqrt(params.mu/(params.Rz+h));
q0 = [0;V;0;h+Rz;0;0];
% От 0 до одного орбитального периода
tk = 2*pi*(h+params.Rz)/V;
% Вызываем функцию интегрирования
[t,q] = ode45(@(t,q) dqdt(t,q,params),q0,[0 tk]);
```

Результат интегрирования

После выполнения строки

```
[t,q] = ode45(@dqdt, q0, [0 tk]);
```

Матрица q будет содержать 6 столбцов и число строк, равное числу шагов интегрирования, соответсвующие моментам времени, содержащимися в столбце t:

$$t = \begin{bmatrix} 0 \\ t_1 \\ t_2 \\ \dots \\ t_n \end{bmatrix}, \quad q = \begin{bmatrix} v_{x0} & v_{y0} & v_{z0} & x_0 & y_0 & z_0 \\ v_{x1} & v_{y1} & v_{z1} & x_1 & y_1 & z_1 \\ v_{x2} & v_{y2} & v_{z2} & x_2 & y_2 & z_2 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ v_{xn} & v_{yn} & v_{zn} & x_n & y_n & z_n \end{bmatrix}.$$

Экспорт результатов в текстовый файл

Для обработки результатов в других программах возможна запись таблицы с результатами в текстовый файл, в котором значения в каждой строке разделяются запятой:

```
dlmwrite('results.csv',[t, q], ',');
```

$$\begin{bmatrix} t_0, & v_{x0}, & v_{y0}, & v_{z0}, & x_0, & y_0, & z_0, \\ t_1, & v_{x1}, & v_{y1}, & v_{z1}, & x_1, & y_1, & z_1, \\ t_2, & v_{x2}, & v_{y2}, & v_{z2}, & x_2, & y_2, & z_2, \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ t_n, & v_{xn}, & v_{yn}, & v_{zn}, & x_n, & y_n, & z_n \end{bmatrix}.$$

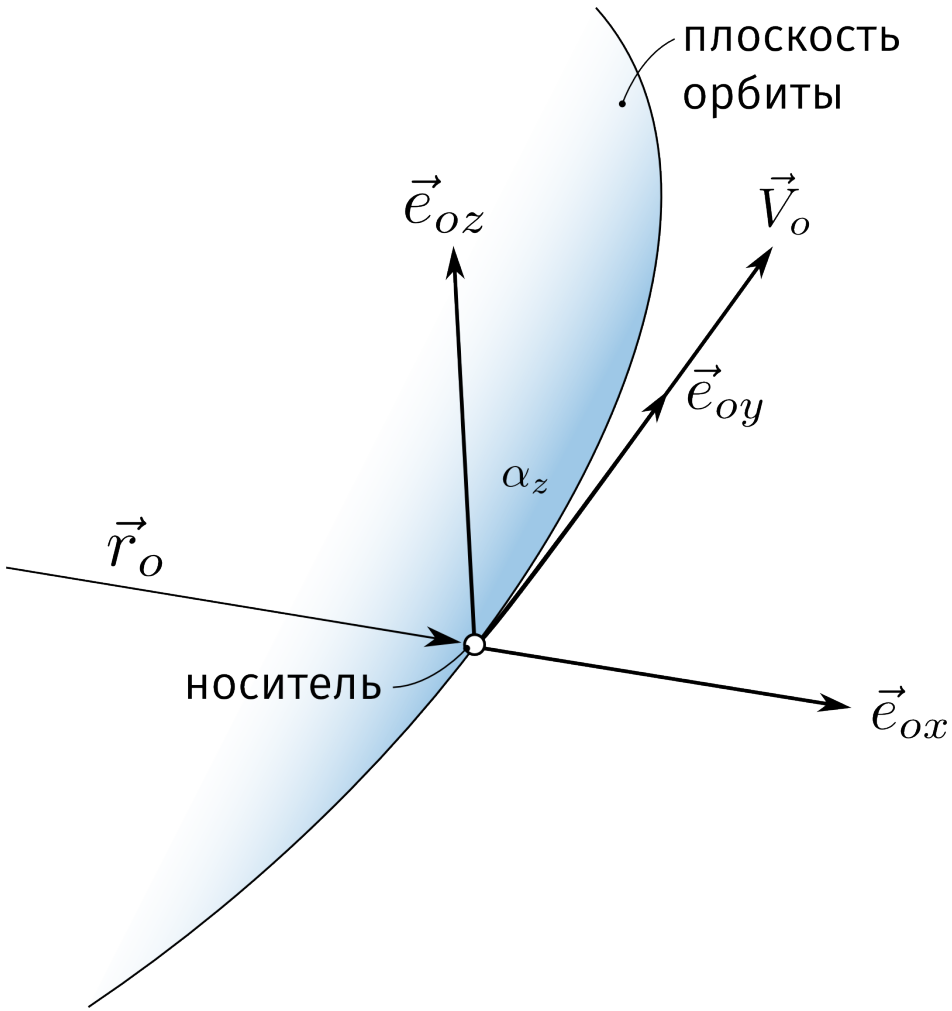
Построение графиков

Файл orbitalMotion.m

```
plot(t, (sqrt(sum(q(4:6,:).^2,2))-Rz)*0.001);
xlabel('t, c');
ylabel('r, км');
```

Пересчёт координат из геоцентрической СК в орбитальную подвижную

Направления осей $Cx_oy_oz_o$



Единичный вектор радиус-вектора носителя:

$$\vec{e}_{ox} = \vec{r}_0 / r_0.$$

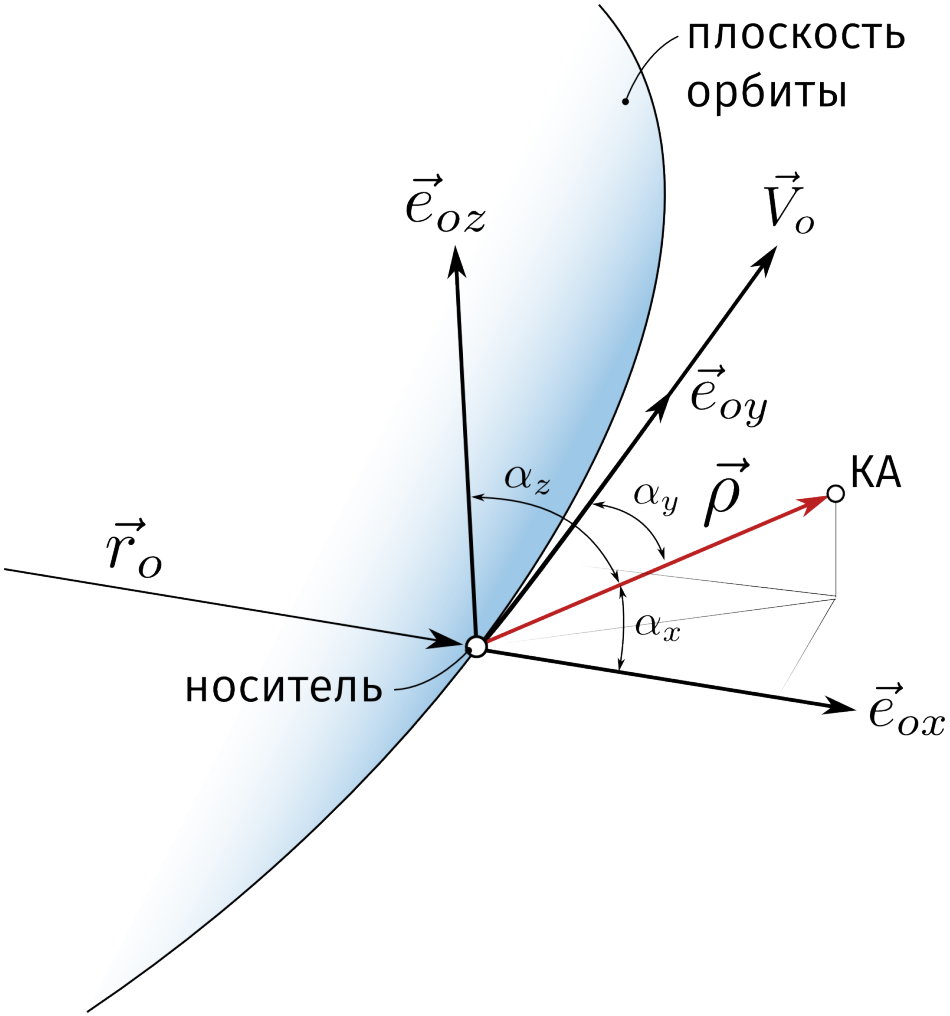
Единичный вектор нормали к орбите:

$$\vec{e}_{oz} = (\vec{r}_0 \times \vec{v}_0)/|\vec{r}_0 \times \vec{v}_0|.$$

Единичный вектор трансверсали:

$$\vec{e}_{oy} = (\vec{e}_{oz} \times \vec{e}_{ox})/|\vec{e}_{oz} \times \vec{e}_{ox}|.$$

Положение КА относительно носителя



Радиус-вектор положения КА относительно носителя:

$$\vec{\rho} = \vec{r} - \vec{r}_0$$

Координаты КА относительно носителя в $Cx_oy_oz_o$:

$$x = \vec{\rho} \cdot \vec{e}_{ox} = \vec{\rho} \cos \alpha_x,$$
$$y = \vec{\rho} \cdot \vec{e}_{oy} = \vec{\rho} \cos \alpha_y,$$
$$z = \vec{\rho} \cdot \vec{e}_{oz} = \vec{\rho} \cos \alpha_z.$$

Вектор относительного положения

Матрица результатов интегрирования абсолютного движения двух материальных точек:

```
q =
v1x, v1y, v1z, x1, y1, z1, v2x, v2y, v2z, x2, y2, z2
v1x, v1y, v1z, x1, y1, z1, v2x, v2y, v2z, x2, y2, z2
...
```

Координатный столбец вектора положения второй точки относительно первой в проекциях на неподвижную систему координат:

```
rho = q(:,10:12)-q(:,1:3);
```

Орты орбитальной подвижной системы

```
q =
v1x, v1y, v1z, x1, y1, z1, v2x, v2y, v2z, x2, y2, z2
v1x, v1y, v1z, x1, y1, z1, v2x, v2y, v2z, x2, y2, z2
...
```

Координатные столбцы единичных векторов орбитальной подвижной системы координат, связанной с первым телом:

```
r = sqrt( sum( q(:,4:6).*q(:,4:6) ,2) );
ex0 = q(:,4:6)./repmat(r,1,3);
```

```
v = sqrt( sum( q(:,1:3).*q(:,1:3) ,2) );
ev0 = q(:,4:6)./repmat(v,1,3);
```

Орты орбитальной подвижной системы

```
q =
v1x, v1y, v1z, x1, y1, z1, v2x, v2y, v2z, x2, y2, z2
v1x, v1y, v1z, x1, y1, z1, v2x, v2y, v2z, x2, y2, z2
...
```

Единичный вектор нормали к плоскости орбиты:

```
ez0 = cross(ex0,ev0,2);
ez0 = ez0./repmat(sqrt(sum(ez0.*ez0,2)),1,3);
```

Единичный вектор трансверсали:

```
ey0 = cross(ez0,er0,2);
ey0 = ey0./repmat(sqrt(sum(ey0.*ey0,2)),1,3);
```

Относительное положение

Координатный столбец вектора положения второй точки относительно первой в проекциях на неподвижную систему координат:

```
rho = q(:,10:12)-q(:,1:3);
```

Тот же вектор в проекция на оси орбитальной подвижной системы:

```
rho_o = [dot(rho,ex0,2),dot(rho,ey0,2),dot(rho,ez0,2)];
```

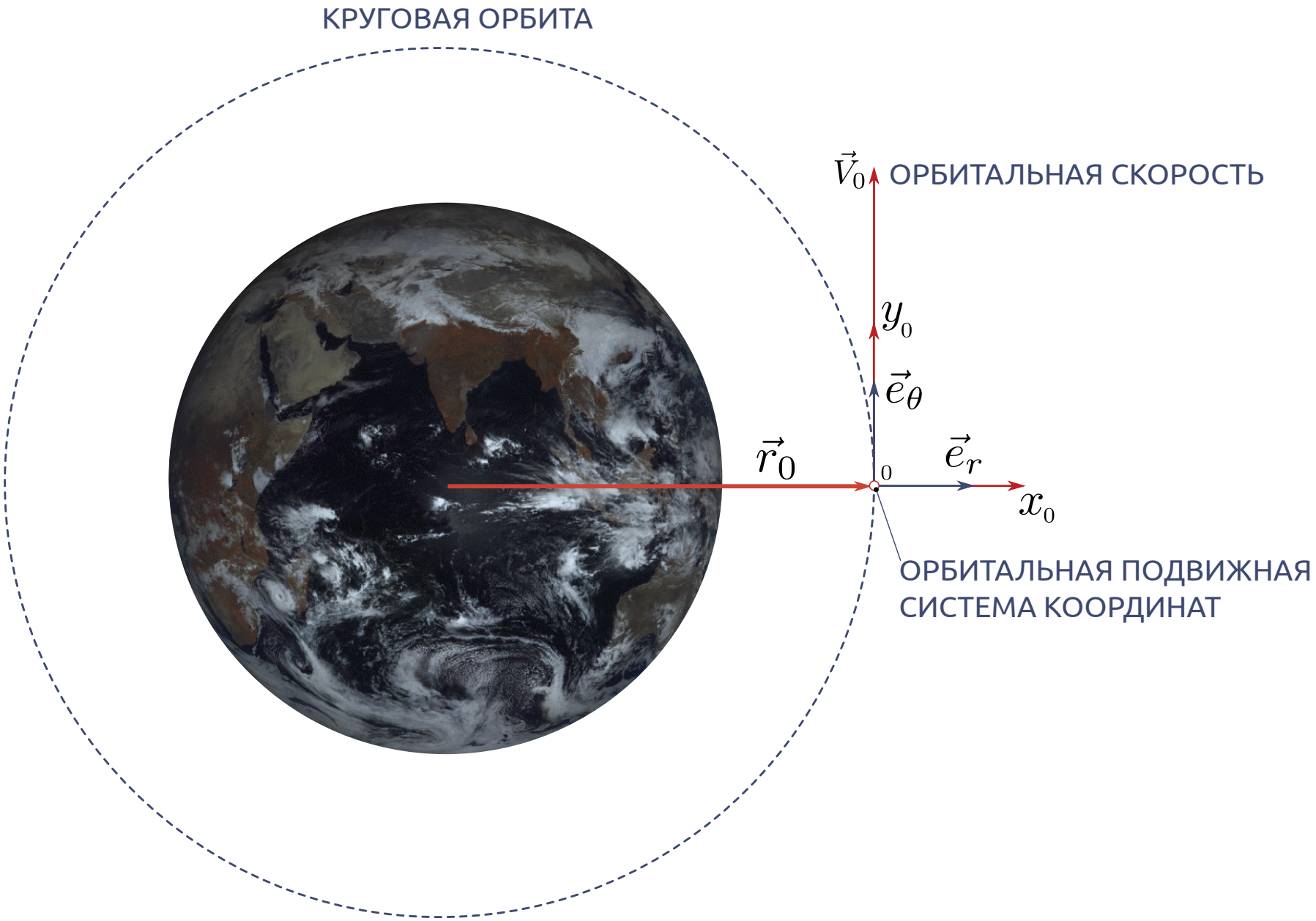
Задание

- Используя программу численного интегрирования движения спутника в центральном поле определить движение спутника-носителя относительно геоцентрической инерциальной системы координат с начальными условиями, соответствующими его движению по круговой орбите высотой 800 км.

Задание

- Второй спутник отделяется от первого со скоростью 1 м/с в направлении вектора орбитальной скорости. Определить движение второго спутника относительно геоцентрической инерциальной системы координат.
- Построить траекторию движения второго спутника относительно первого в орбитальной подвиженой системе координат $Ox_oy_oz_o$. Начало орбитальной подвижной системы координат расположеное в центре масс первого спутника (точка O).

Орбитальная подвижная СК

- Ось Ox_o направлена из вдоль радиус-вектора первого спутника относительно центра Земли (“вверх”).
- Ось Oy_o лежит в плоскости орбиты и направлена в сторону орбительной скорости первого спутника.
- Ось Oz_o дополняет систему до правой.

Орбитальная подвижная СК